

## Document Signature SaaS (Mini DocuSign)

### Project Understanding Document (Final Approved Version)

---

#### 1. Project Overview

The Document Signature SaaS is a web-based document workflow platform that enables users to upload PDF documents, assign signing fields, invite multiple signers, track signing progress, generate audit trails, and produce finalized signed PDFs.

The system is inspired by products such as DocuSign but intentionally limited to a Version 1 scope that can be realistically implemented while still demonstrating professional software engineering practices.

This project is not a PDF editor.

This project is primarily a workflow-management and document-approval platform where PDF signing is one component of a larger system.

The project focuses heavily on:

- Authentication
- Authorization
- Business workflows
- Document lifecycle management
- Multi-user collaboration
- Audit logging
- Notifications
- PDF generation
- Secure file handling

The goal is to build a portfolio-quality SaaS application that resembles how enterprise document-signing systems are designed.

---

#### 2. Problem Statement

Many organizations still use manual document-signing processes:

Send PDF

↓

Print

↓

Sign

↓

Scan



Send Back

This creates several problems:

- Slow approval cycles
- Lost documents
- Poor visibility
- No centralized tracking
- No audit history
- Difficult coordination among multiple signers
- High administrative overhead

The platform replaces this process with:

Upload



Assign Signers



Send Invitations



Collect Signatures



Track Progress



Generate Final PDF



Maintain Audit Trail

---

### **3. Target Users**

#### **Individual Professionals**

Examples:

- Freelancers
- Consultants
- Trainers
- Contractors

Use Cases:

- Client agreements
  - Project approvals
  - Service contracts
-

## **Small Businesses**

Examples:

- Agencies
- Startups
- Service providers

Use Cases:

- Vendor contracts
  - Employee onboarding
  - Internal approvals
- 

## **Internal Teams**

Examples:

- HR departments
- Legal teams
- Operations teams

Use Cases:

- Offer letters
  - NDAs
  - Compliance documents
  - Internal authorization forms
- 

## **4. User Roles**

### **Owner**

The creator and manager of a document workflow.

Capabilities:

- Register
- Login
- Upload PDFs
- View owned documents
- Delete draft documents
- Add signers
- Remove signers
- Assign fields
- Send invitations
- View audit logs
- Download completed documents
- Configure expiration dates

- Extend expiration dates
  - Resend invitations
  - Participate as a signer
- 

### **Signer**

A user invited to sign a document.

Capabilities:

- Register
  - Login
  - Access assigned documents
  - View entire document
  - Fill assigned fields
  - Submit signature
  - Reject document
  - Provide mandatory rejection reason
  - View audit trail
  - Download final PDF (if permitted)
- 

## **5. Core Business Rules**

### **Rule 1**

Only authenticated users may sign documents.

No anonymous signing.

---

### **Rule 2**

Signing links expire.

Expired links become unusable.

Owners may extend expiration.

---

### **Rule 3**

Documents become immutable after signing begins.

After the first signature:

- No PDF replacement
  - No signer additions
  - No signer removal
  - No field modifications
- 

### **Rule 4**

Maximum 15 signers per document.

---

#### **Rule 5**

Only PDF files are accepted.

---

#### **Rule 6**

Typed signatures only.

Version 1 does not support:

- Drawn signatures
- Uploaded signature images
- Cryptographic signatures

Each signature consists of:

- Typed name
- Selected font

The system stores:

- Typed name
- Font choice

The PDF rendering is generated from these values.

---

#### **Rule 7**

Parallel Signing Workflow

All invited signers may sign immediately.

No sequential approval chain exists in Version 1.

---

#### **Rule 8**

All assigned fields are mandatory.

A signer cannot submit until every assigned field is completed.

---

#### **Rule 9**

Rejection Reason Required

A signer who rejects must provide a reason.

Requirements:

- Mandatory
  - Visible to document owner
  - Recorded in audit logs
- 

#### **Rule 10**

Document Rejection Ends Workflow

If any signer rejects:

Document Status = REJECTED

Consequences:

- Workflow terminates
  - No final PDF generated
  - No additional signatures accepted
  - Owner must create a new document if changes are required
- 

## **Rule 11**

Owner Is Treated As Normal Signer

If the owner must sign:

- Owner receives signer assignment
  - Owner follows same signing process
  - No special signing privileges
- 

## **6. Functional Requirements**

### **Authentication Module**

Features:

- User Registration
  - User Login
  - JWT Authentication
  - Password Hashing
  - Protected Routes
  - Logout
  - Refresh Token Support (optional)
- 

### **User Profile Module**

Features:

- Update profile
  - Change password
  - Choose preferred signature font
  - Store signature preferences
- 

### **Document Module**

Features:

- Upload PDF

- View document
- Download document
- Delete draft
- View status
- Extend expiration
- Download final PDF

---

## **Signer Management Module**

Features:

- Add signers
- Remove signers
- Invite signers
- Resend invitations

Behavior:

If invited email already belongs to an existing user:

Assignment is linked directly.

If email does not exist:

Pending invitation remains until registration.

---

## **Field Management Module**

Supported Fields:

### **Signature Field**

Stores signature placement.

### **Date Field**

Stores signing date.

### **Name Field**

Stores signer name.

### **Custom Text Field**

Stores custom user input.

Owners assign fields to specific signers.

---

## **Signing Module**

Features:

- View document
- View assigned fields
- Fill fields
- Submit signature

- Reject document

---

## **Audit Module**

Tracks:

- User registered
- Document created
- Document uploaded
- Signer added
- Invitation sent
- Reminder sent
- Document viewed
- Field completed
- Document signed
- Document rejected
- Link expired
- Document completed

Audit logs are immutable.

---

## **Notification Module**

Email notifications for:

- Invitation sent
- Reminder sent
- Document signed
- Document rejected
- Document completed
- Document expired

---

## **7. Non-Functional Requirements**

### **Security**

Requirements:

- Password hashing
  - JWT authentication
  - Authorization checks
  - Protected APIs
  - Input validation
  - Secure file access
-



## **Reliability**

Requirements:

- No signature loss
  - No audit loss
  - Consistent status transitions
  - Transaction-safe updates
- 

## **Maintainability**

Architecture must support:

- Additional field types
  - Cloud storage migration
  - New workflows
  - Future integrations
- 

## **Performance**

Targets:

- Fast uploads
  - Dashboard loads quickly
  - API responses generally under 500ms
  - Efficient PDF processing
- 

## **8. Complete User Journey**

### **Owner Flow**

Register



Login



Upload PDF



Place Fields



Add Signers



Send Invitations



Monitor Progress



Receive Notifications



Download Completed PDF

---

### **Signer Flow**

Receive Invitation



Open Link



Register/Login



View PDF



Fill Fields



Sign or Reject



Submit



Receive Completion Notification

---

### **9. Document Lifecycle**

DRAFT



PENDING



PARTIALLY\_SIGNED



COMPLETED

Alternative Paths:

PENDING



REJECTED

or

PENDING



EXPIRED

---

## 10. Status State Machine

Valid States:

- DRAFT
- PENDING
- PARTIALLY\_SIGNED
- COMPLETED
- REJECTED
- EXPIRED

Invalid transitions must be blocked.

Examples:

Not Allowed:

COMPLETED → PENDING

COMPLETED → DRAFT

REJECTED → PARTIALLY\_SIGNED

---

## 11. System Workflow

Owner Uploads PDF



Document Stored



Fields Created



Signers Added



Invitations Sent



Signers Access Document



Fields Completed



Audit Logs Generated



All Signatures Collected



Final PDF Generated

↓

Status = COMPLETED

---

## 12. High-Level Data Flow

User

↓

Frontend

↓

FastAPI API Layer

↓

Service Layer

↓

Repository Layer

↓

Database

And

Service Layer

↓

File Storage

---

## 13. Database Design Strategy

Major Entities:

### Users

Stores:

- Profile
  - Authentication data
  - Preferences
- 

### Documents

Stores:

- Metadata
  - Status
  - Ownership
  - Expiration
- 

### Document Files

Stores:

- Original file path
  - Final file path
- 

## **Document Signers**

Stores:

- Assignments
  - Invitation status
  - Signing status
- 

## **Signature Fields**

Stores:

- Coordinates
  - Page number
  - Field type
  - Assigned signer
- 

## **Field Values**

Stores:

- User-provided values
  - Completion timestamps
- 

## **Audit Logs**

Stores:

- Action
  - Actor
  - Timestamp
  - Metadata
- 

## **Notifications**

Stores:

- Email history
  - Delivery status
- 

## **Signing Tokens**

Stores:

- Token hashes
- Expiration

- Usage metadata

---

## **14. Security Model**

### **Authentication**

JWT-based authentication.

---

### **Authorization**

Owners:

Access only owned documents.

Signers:

Access only assigned documents.

---

### **File Protection**

Documents are never exposed directly.

Every file request passes through authorization checks.

---

### **Audit Protection**

Audit records cannot be edited or deleted.

---

## **15. Edge Cases**

### **Signer Never Registers**

Solutions:

- Reminders
- Expiration
- Resend invitation

---

### **Signer Rejects**

Status becomes:

REJECTED

Workflow ends.

---

### **Link Expiration**

Status becomes:

EXPIRED

Owner may extend expiration.

---

### **Multiple Link Opens**

Allowed.

Must remain idempotent.

---

### **Partial Signing**

Supported.

Document remains:

PARTIALLY\_SIGNED

until all required signers finish.

---

## **16. Scalability Considerations**

Version 1:

- Single backend
- Single PostgreSQL database
- Local file storage

Future Growth:

- AWS S3
- Redis
- Celery
- Background workers
- CDN
- Object storage

Architecture must allow migration without major rewrites.

---

## **17. Recommended Architecture**

Layered Architecture

API Layer

↓

Service Layer

↓

Repository Layer

↓

Database

Never:

Route

↓

Database

Direct access is prohibited.

Business logic belongs in services.

---

## **18. Recommended Technology Stack**

Backend:

- Python
- FastAPI
- SQLAlchemy
- PostgreSQL
- Alembic

Authentication:

- JWT

Validation:

- Pydantic

PDF Processing:

- PyMuPDF

Email:

- Mailtrap (Development)
- Resend (Production)

Frontend:

- React
- Vite
- TailwindCSS
- React PDF

Version Control:

- Git
  - GitHub
- 

## **19. Folder Structure Philosophy**

Feature-Oriented Organization

Example Modules:

- auth
- users
- documents
- signers
- signatures
- audit
- notifications



Each module owns:

- routes
- schemas
- services
- repositories

Avoid large global folders that mix unrelated features.

---

## **20. Major Technical Challenges**

- PDF coordinate mapping
  - Signature rendering
  - Authorization enforcement
  - Audit consistency
  - State transition validation
  - Multi-signer workflow management
  - Expiration handling
  - Secure file delivery
- 

## **21. Success Criteria**

Functional Success:

- Upload works
- Invitations work
- Multi-signer workflow works
- Signing works
- Rejection works
- Audit logging works
- Email notifications work
- Final PDF generation works

Technical Success:

- Clean architecture
- Secure authentication
- Proper authorization
- Modular codebase
- Well-designed database
- Production-style API design

Portfolio Success:

A reviewer should immediately recognize experience with:

- Authentication

- Authorization
- File handling
- PDF processing
- Workflow engines
- Audit systems
- Email systems
- Business logic modeling
- SaaS architecture

This should feel like a real business application rather than a CRUD project.